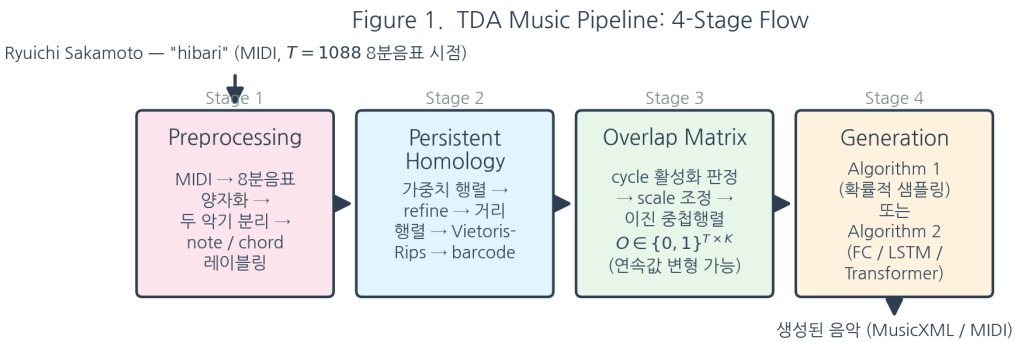
5. 시각자료 (Figures)

본 장은 논문 본문에 삽입될 정적 figure 6개와 보충자료(supplementary material)로 제공되는 1개의 interactive figure를 모아서 제시한다.

본 장에 등장하는 시각자료의 개요

#	내용	배치
1	4-stage 파이프라인 흐름도	본문
7	Inst 2의 모듈 구조 + 쉼표 시각화 (모듈 확대 포함)	본문
2	발견된 cycle의 3D interactive 시각화 (재생 가능)	**보충자료**
3	Tonnetz 격자에 hibari 음 배치	본문
4	Persistence barcode 다이어그램	본문
5	원곡 vs 생성곡 piano roll 비교 (3 구간)	본문
6	학습 epoch별 JS divergence 곡선	본문

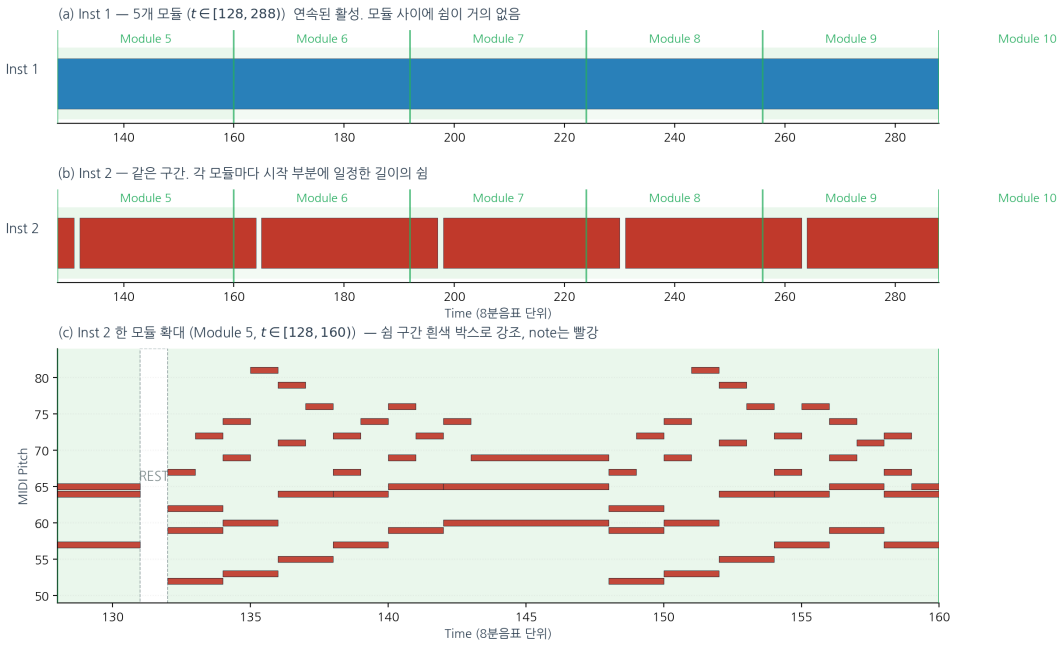
Figure 1 — TDA Music Pipeline: 4-Stage Flow



캡션. 본 연구의 파이프라인은 네 개의 순차적 단계로 구성된다. **Stage 1**은 MIDI 원본을 8분음표 단위로 양자화하고 두 악기를 분리한 뒤 note / chord 레이블링을 수행한다. **Stage 2**는 note 간 가중치 행렬을 구축하고 refine 과정을 거쳐 거리 행렬을 만든 뒤, Vietoris-Rips 복합체로부터 1차 persistent homology를 계산한다. **Stage 3**은 발견된 모든 cycle의 활성화 정보를 시점별 이진 중첩행렬 $O \in \{0,1\}^{T \times K}$로 변환한다(연속값 변형도 정의되어 있으나 본 실험에서는 이진 형태가 주로 사용되었다). **Stage 4**는 이 중첩행렬을 seed로 하여 Algorithm 1 (확률적 샘플링) 또는 Algorithm 2 (FC / LSTM / Transformer 신경망)로 새로운 음악을 생성한다.

Figure 7 — 두 악기의 모듈 반복 (Inst 1은 연속, Inst 2는 모듈마다 쉼)

Figure 7. 두 악기의 모듈 반복 — Inst 1은 연속, Inst 2는 모듈마다 쉼



캡션. 본 figure는 hibari의 두 악기가 **같은 32-timestep 모듈 구조를 반복**하지만, 활성/쉼 패턴의 성격이 근본적으로 다름을 시각적으로 드러낸다. 개별 note는 (a)와 (b)에서 의도적으로 표시하지 않는다. 중요한 것은 "언제 악기가 울리고 언제 쉬는가"의 거시적 패턴이기 때문이다. 셋 panel 모두 같은 시간축 (8분음표 단위)을 공유한다. **모듈 번호는 두 악기 모두 동일한 절대 시간 기준으로 붙이므로** — inst 1과 inst 2는 처음부터 같은 박자 위에서 진행된다 — Module 5는 inst 1과 inst 2 양쪽 모두에서 $t \in [128, 160)$을 가리킨다.

- **(a) Inst 1 — Modules 5~9 ($t \in [128, 288)$)** — 5개 모듈의 활성 구간을 파란 띠로 표시하였다. 진한 초록 수직선은 모듈 경계, 옅은 초록 배경은 모듈 영역이다. 이 5개 모듈 구간 동안 inst 1의 활성 띠에는 **쉼(흰 공백)이 전혀 나타나지 않는다**. 실제로 전체 $T = 1056$ timestep에 걸쳐 inst 1의 쉼 구간은 0 timestep이다 — inst 1은 처음부터 끝까지 쉬지 않고 연주한다.
- **(b) Inst 2 — 같은 구간 ($t \in [128, 288)$)** — 같은 시간축에 inst 2의 활성을 빨간 띠로 표시하였다. 진한 초록 수직선과 옅은 초록 배경은 (a)와 동일한 모듈 경계를 나타낸다. inst 1과 달리 각 모듈의 초반부마다 일정한 길이의 흰 공백 — 즉 **쉼** — 이 규칙적으로 나타난다. 전체 $T = 1056$ timestep 중 inst 2의 쉼 구간은 총 64 timestep이며, 대부분 모듈 경계 직전/직후에 집중되어 있다. 두 악기가 같은 모듈 길이를 공유하면서도 rest의 존재 여부에서 근본적으로 다른 역할을 맡고 있음이 드러난다.
- **(c) Inst 2 한 모듈 확대 (Module 5, $t \in [128, 160)$)** — inst 2의 한 모듈 안을 piano roll로 확대하여, 모듈 내 쉼 구조를 자세히 관찰한다. 빨간 사각형은 inst 2의 실제 note이며, 쉼이 모듈의 어디에 어떤 길이로 배치되는지 자세히 볼 수 있다.

이 figure가 본 연구에서 중요한 이유는 두 가지이다.

1. **두 악기 역할 분담의 시각적 확인.** inst 1이 시간 방향으로 "끊김 없이 흘러가는 기저 흐름"을 담당하고 inst 2가 그 위에 "의도적으로 쉼을 두며 얹히는" 역할을 맡는 구조가 figure에서 직접 확인된다. 이는 §2.9에서 분석한 두 악기의 상보적 역할과 일치한다.
2. **Intra / Inter 가중치 분리의 정당화.** 본 연구가 intra (각 악기 내부) / inter (두 악기 사이의 시차 결합) / simul (동시 타건)을 분리한 것은 이러한 **두 악기의 역할 차이**를 보존하기 위한 필연적 선택임을 figure가 시각적으로 정당화한다.

Figure 2 (Interactive Supplementary) — hibari Cycles in 3D, with Click-to-Play

> 본 figure는 정적 PNG 형태로는 제공되지 않으며, 별도의 interactive HTML 파일로 제공된다 (제출 시 부록 디스크 또는 URL로 동봉).

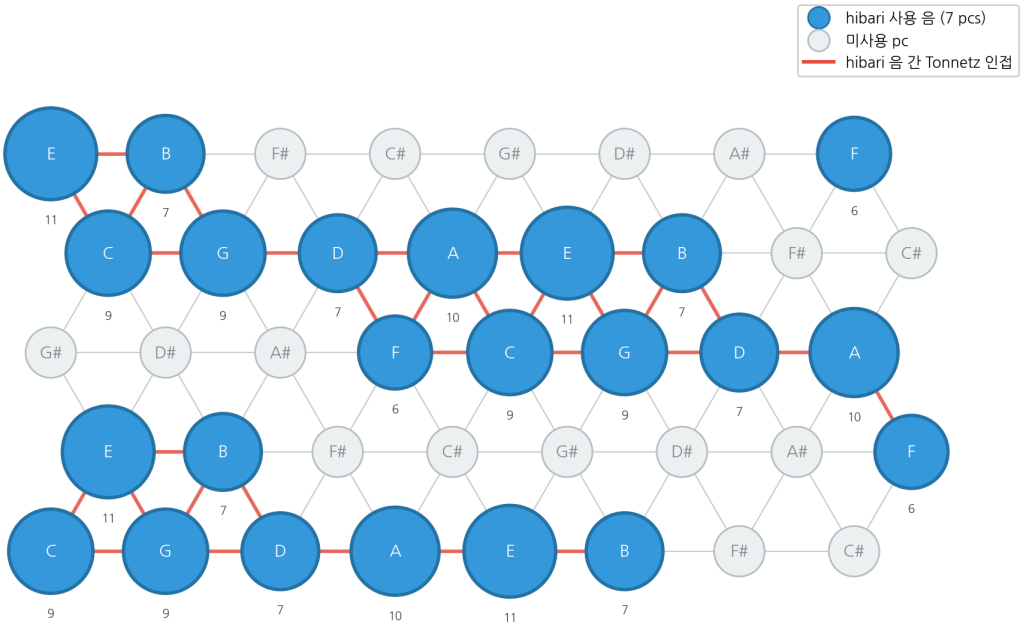
interactive HTML에서 가능한 조작.

- 마우스 드래그로 자유 회전, 스크롤로 줌
- 우측 legend 클릭으로 특정 cycle on/off
- cycle 선에 마우스를 올리면 그 cycle이 포함하는 모든 note의 pitch 이름 + 옥타브 + duration이 hover로 표시됨 (예: `label 19: C5 (dur=1)`)
- 화면 하단의 **"오디오 켜기"** 버튼을 한 번 클릭하면 브라우저 오디오 컨텍스트가 활성화되고, 이후 **cycle 선을 클릭하면** 해당 cycle을 구성하는 note들이 차례대로 sine wave로 재생됨 (Tone.js 기반, MIDI pitch → 주파수 변환). 이를 통해 "위상학적으로 한 cycle을 이루는 note들이 음악적으로 어떻게 들리는가"를 직접 청취할 수 있다.

상위 20개 cycle을 색상별로 구분하며, 카메라는 처음부터 위에서 내려다보는 각도로 설정되어 있다. 본 figure의 cycle은 **Tonnetz hybrid distance** ($\alpha = 0.5$)로 계산된 것이다.

Figure 3 — Tonnetz 격자 위 hibari의 pitch class

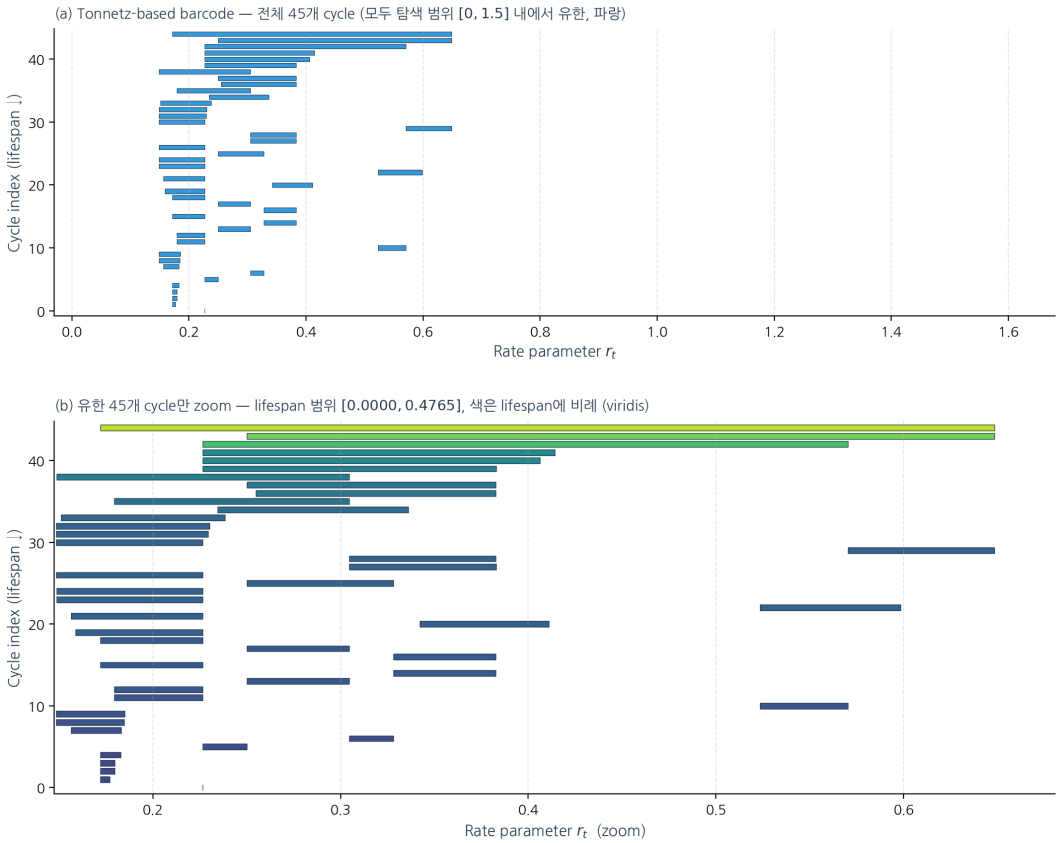
Figure 3. Tonnetz 격자에 배치된 hibari의 7개 pitch class (C major scale)
빨간 선 = Tonnetz 그래프 상의 직접 인접, 숫자 = 원곡 내 출현 횟수



캡션. 12개의 pitch class가 배치된 Tonnetz 격자(가로 = 완전 5도, 대각선 = 장 / 단 3도) 위에서 hibari가 실제로 사용하는 7개 pitch class — C, D, E, F, G, A, B (C major scale)를 진한 파랑으로 강조하였다. 각 노드의 크기는 곡 전체에서 해당 pitch class의 출현 빈도(두 악기 합산)에 비례하며, 빨간 선은 사용된 pitch class 쌍 중 Tonnetz 그래프 상에서 직접 인접한(거리 1) 관계를 나타낸다. 사용된 음들이 격자 위에서 하나의 연결 성분을 이루며 집중되어 있음을 확인할 수 있는데, 이는 hibari가 음악이론적으로 잘 정의된 토너츠 영역 안에서 작동하는 곡이며, 따라서 Tonnetz 거리 함수가 frequency 거리보다 이 곡의 위상 구조를 더 잘 포착한다는 §4.1의 결과와 일관된다.

Figure 4 — Persistence Barcode Diagram (Tonnetz distance)

Figure 4. Persistence Barcode — hibari H_1 cycles (Tonnetz hybrid distance, $\alpha = 0.5$, timeflow weight)



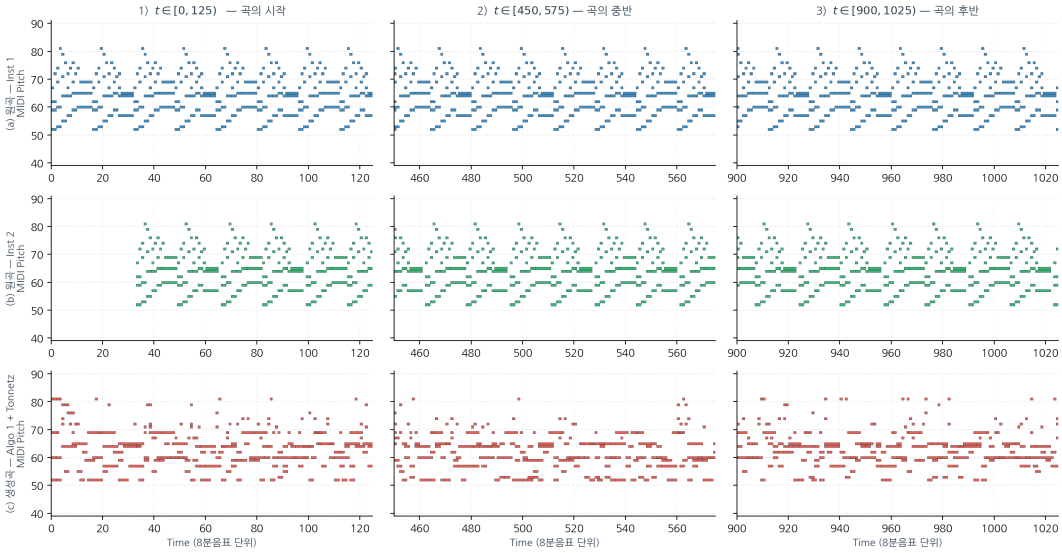
데이터 출처. 본 연구의 주 거리 함수는 **Tonnetz hybrid distance** ($\alpha = 0.5$) 이므로, barcode 역시 Tonnetz 기반으로 계산한다. 구체적으로는 `topology.generate_barcode_numpy`를 rate $r_t \in [0, 1.5]$의 0.01 간격 grid에서 호출하여 H_1 persistence를 직접 재계산한 뒤, 결과를 `docs/figures/fig4_tonnetz_persistence.pkl`에 캐시하였다 (계산 시간 약 2~3분, 총 45개 cycle).

캡션. hibari의 H_1 cycle 전체 45개에 대한 Tonnetz 기반 persistence barcode를 두 panel로 나누어 보인다.

- **(a) 전체 barcode** — 가로축은 rate parameter r_t, 각 막대는 한 cycle의 $[\mathrm{birth}, \mathrm{death}]$ 구간이다. Tonnetz 거리에서는 frequency와 달리 **45개 cycle 전부가 탐색 범위** $[0, 1.5]$ **내에서 유한**하다 (즉 탐색 범위를 벗어나는 "영속 cycle"이 하나도 없다). 위쪽 몇 개 cycle은 lifespan이 0.4를 넘을 정도로 매우 안정적이며, 아래로 갈수록 lifespan이 급격히 짧아진다.
- **(b) 유한 cycle 전체 zoom** — 짧은 lifespan 쪽 cycle들도 식별할 수 있도록 x축을 실측 범위 $[0.00004, 0.47654]$로 조정하였다. 색은 viridis colormap으로 lifespan에 비례한다. 가장 긴 cycle과 가장 짧은 cycle 사이에 약 $10{,}000$배 이상의 lifespan 차이가 있어, Tonnetz 기반 filtration이 hibari에서 구조적으로 중요한 소수 cycle을 매우 선명하게 분리함을 확인할 수 있다.

Figure 5 — Piano Roll 비교 (3 구간)

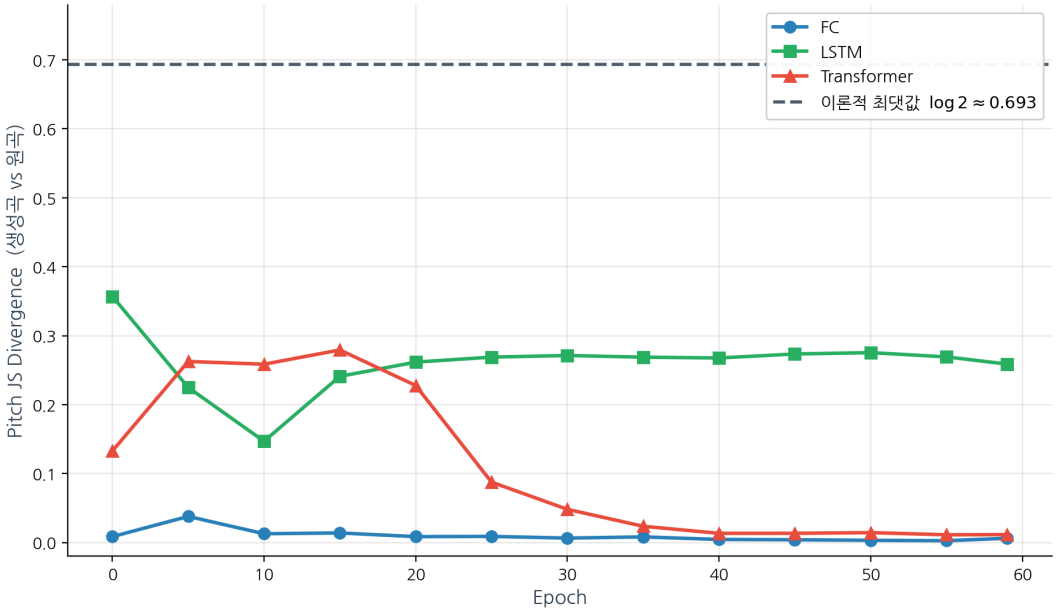
Figure 5. Piano Roll 비교 — 원곡 vs Tonnetz 기반 Algorithm 1 생성곡 (3개 구간 × 125 timesteps)



캡션. 위에서부터 순서대로 (a) 원곡 악기 1, (b) 원곡 악기 2, (c) Algorithm 1 + Tonnetz (K=46, seed=42)로 생성된 곡이다. 전체 $T = 1088$ timesteps을 곡의 시작 ($t \in [0, 125)$), 중반 ($t \in [450, 575)$), 후반 ($t \in [900, 1025)$) 세 구간으로 나누어 각 125 timesteps 씩 나란히 보여준다. 가로축은 시간(8분음표 단위), 세로축은 MIDI pitch이다. 각 구간에서 생성곡의 pitch 분포와 시간적 밀도가 원곡과 유사한 영역에서 활성화되고 있음을 시각적으로 확인할 수 있으며, 이는 3.1절의 JS divergence 정량 결과(0.0398)와 일관된다.

Figure 6 — 학습 Epoch별 JS Divergence 곡선

Figure 6. 학습 epoch별 JS Divergence — FC / LSTM / Transformer
(Tonnetz 기반 overlap, 동일 초기 seed)



캡션. FC / LSTM / Transformer 세 모델을 동일한 데이터(Tonnetz overlap)와 동일 seed에서 60 epoch 학습시키면서, 매 5 epoch마다 생성곡을 만들어 원곡과의 pitch JS divergence를 측정한 결과이다. FC 모델이 epoch 30 근처에서 $\mathrm{JS} \approx 0.003$에 도달하여 가장 빠르게 수렴하고, Transformer는 epoch 30 이후 같은 수준까지 따라잡는다. 반면 LSTM은 $\mathrm{JS} \approx 0.26$ 근처에서 머물며 추가 개선이 일어나지 않는다. 이는 3.4절의 해석 — "hibari는 out of noise 앨범의 곡으로서 시간 문맥보다 음의 공간적 배치가 핵심이며, 시간 인과를 강하게 모델링하는 LSTM은 오히려 곡의 미학과 어긋난다" — 와 일관된 관찰이다.

재현 방법

(전공자 / 코드 재현이 필요한 독자만 참조. 인쇄 제출본에는 본 절을 넣지 않는다.)

```
cd tda_pipeline/docs/figures

#    figures (      )
python make_fig1_pipeline.py        #
python make_fig3_tonnetz_hibari.py  #
python make_fig4_barcode.py         # pickle/h1_rBD_*.pkl
python make_fig5_pianoroll.py       #       + cache    (~5s)
python make_fig6_js_curve.py        # fig6_curves.json        ,
                                    #            (~2-3 min)
python make_fig7_inst2_modules.py   #           (~3s)

# Interactive supplementary
python make_fig2_interactive.py     # Plotly + Tone.js HTML
```

모든 정적 figure는 200 dpi PNG로 저장된다. 한글 폰트는 `_fontsetup.py`에서 NanumGothic을 자동 등록한다. Interactive figure는 `plotly` 패키지가 필요하며, 생성된 HTML은 외부 CDN에서 plotly.js와 Tone.js를 로드하므로 처음 열 때 인터넷 연결이 필요하다.